

Base de Datos No SQL

SEMANA 14

Heider Sanchez
hsanchez@utec.edu.pe

3.



Base de Datos NoSQL

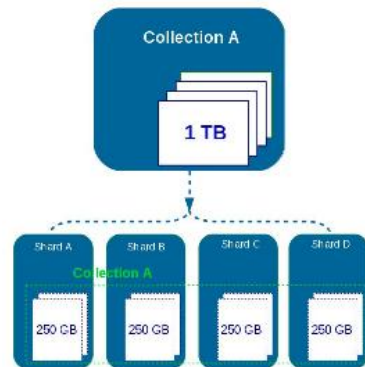
BD Orientado a Documentos

BD de Documentos

Características:

- **Modelo basado en documentos:** Almacenan datos en formato JSON, BSON o XML en lugar de tablas tradicionales.
- **Estructura flexible:** No requieren un esquema fijo, lo que permite cambios sin afectar toda la base de datos.
- **Escalabilidad horizontal:** Diseñadas para manejar grandes volúmenes de datos distribuidos.
- **Alta velocidad de lectura/escritura:** Son más eficientes en consultas sobre documentos específicos.
- **Indexación avanzada:** Permiten búsquedas rápidas dentro de documentos sin la complejidad de múltiples relaciones.
- **Integración con APIs modernas:** Ideales para aplicaciones web y móviles que trabajan con datos dinámicos.

```
{  
  "_id": "5cf0029cafff5056591b0ce7d",  
  "firstname": "Jane",  
  "lastname": "Wu",  
  "address": {  
    "street": "1 Circle Rd",  
    "city": "Los Angeles",  
    "state": "CA",  
    "zip": "90404"  
  },  
  "hobbies": ["surfing", "coding"]  
}
```



BD de Documentos

Ejemplos de JSON

```
{  
  "nombre": "Juan",  
  "edad": 30,  
  "ciudad": "Madrid",  
  "profesión": "Ingeniero"  
}
```

```
{  
  "persona": {  
    "nombre": "Ana",  
    "edad": 28,  
    "dirección": {  
      "calle": "Gran Vía",  
      "número": 12,  
      "ciudad": "Barcelona"  
    },  
    "contacto": {  
      "teléfono": "123456789",  
      "email": "ana@email.com"  
    }  
  }  
}
```

```
{  
  "texto_id": 101,  
  "contenido": "La inteligencia  
artificial está revolucionando el  
mundo.",  
  "autor": "Heider",  
  "fecha_creacion": "2025-04-07",  
  "categoria": "Tecnología",  
  "idioma": "es",  
  "embedding": [0.21, -0.45, 0.78,  
0.36, -0.12, 0.67, -0.34, 0.89]  
}
```

BD de Documentos

Algunos casos de uso:

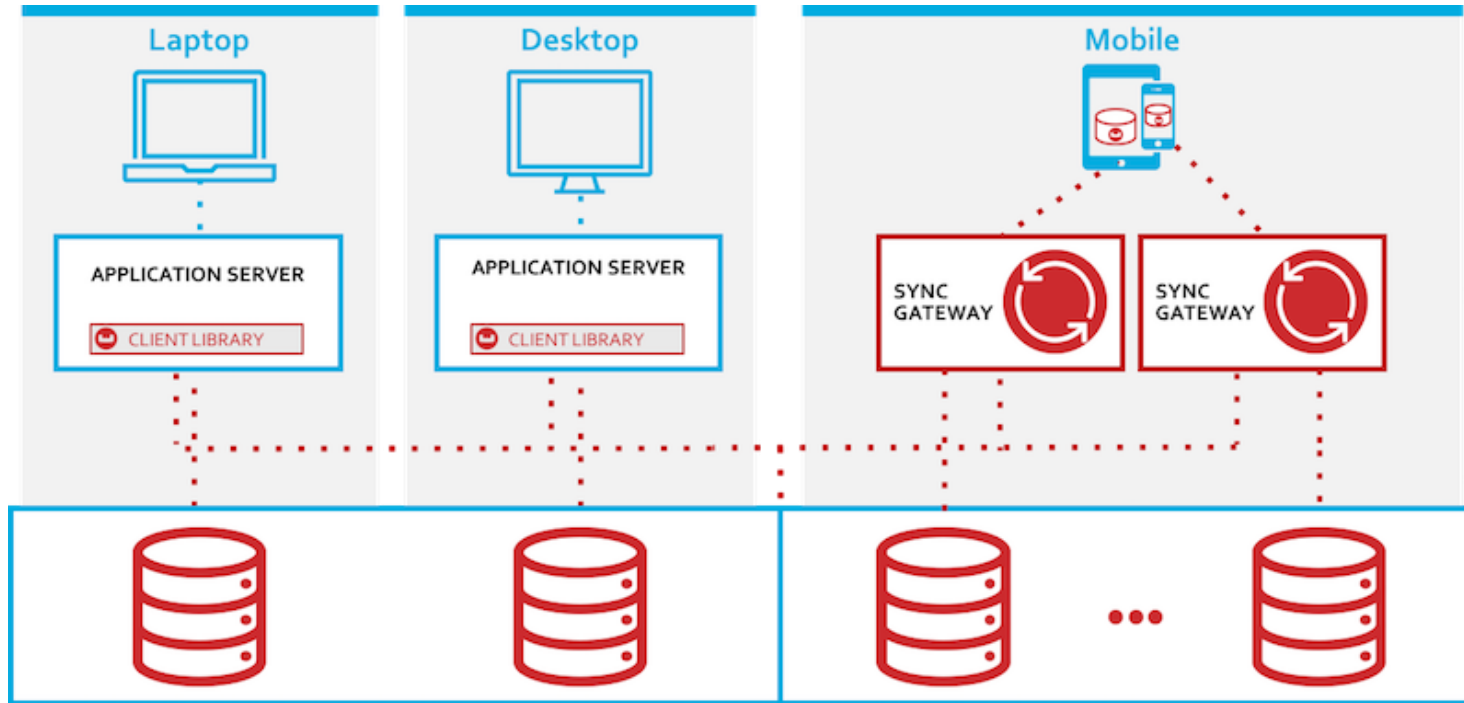
- **Aplicaciones web y móviles:** Almacenan datos JSON para APIs y sistemas dinámicos. Soportan sincronización rápida y acceso eficiente a datos para apps con alta interacción.
- **E-Commerce:** Permite almacenar productos con atributos variables (colores, tallas, precios, etc.) sin esquema fijo.
- **Sistemas de gestión de contenido (CMS):** Flexibilidad en almacenamiento de artículos, páginas o bloques con estructuras flexibles.
- **Redes sociales:** Manejo de publicaciones, perfiles de usuario, historiales y mensajes con estructuras anidadas y dinámicas.
- **Big Data y analítica:** Procesamiento eficiente de grandes volúmenes de información.
- **Inteligencia artificial:** Almacenamiento de datasets para **machine learning**.



BD de Documentos: Productos

Base de Datos	Año de Creación	Empresa	Escalabilidad	Características claves	Casos de Uso
MongoDB	2009	MongoDB Inc.	Escalabilidad horizontal con sharding y replicación	JSON/BSON, agregaciones, índices avanzados, Atlas (DBaaS)	Aplicaciones web, catálogos, IoT, sistemas CRM
CouchBase	2010	Couchbase Inc.	Escalabilidad horizontal nativa	Motor híbrido memoria-disco, N1QL (SQL-like), indexación flexible	Aplicaciones en tiempo real, móviles, analítica
Amazon DocumentDB	2019	Amazon Web Services	Escalabilidad automática en la nube (AWS)	Compatible con API de MongoDB, servicio gestionado, cifrado integrado	Aplicaciones en AWS, reemplazo de MongoDB
Firebase Firestore	2017	Google (Alphabet)	Escalabilidad automática global (multi-region)	Realtime updates, sincronización, SDKs móviles, almacenamiento cloud-native	Aplicaciones móviles, chats, apps colaborativas

BD de Documentos : CouchBase



Sync | Couchbase Docs

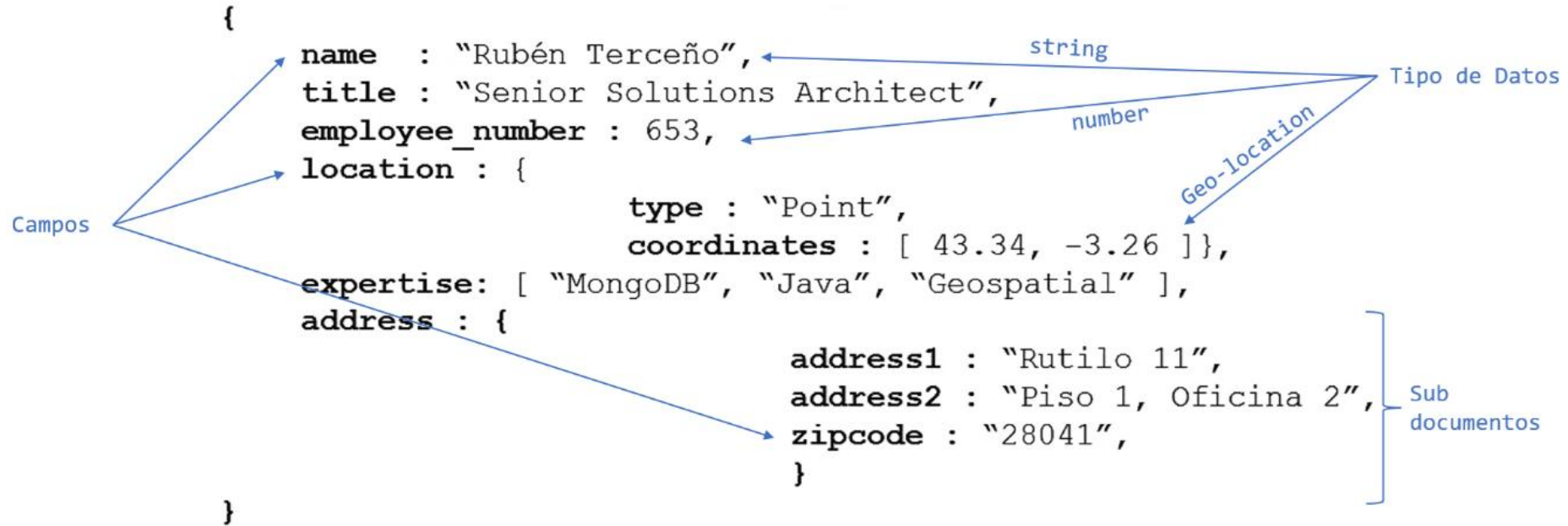
BD de Documentos : MongoDB



- MongoDB es una base de datos NoSQL orientada a documentos, de código abierto, que almacena datos en formato **BSON** (una extensión de JSON).
- Fue diseñada para ofrecer alta **escalabilidad, flexibilidad y rendimiento**.
- **Características claves:**
 - **Altamente escalable:** soporta sharding y replicación
 - **Consultas avanzadas** con agregaciones y filtros
 - **Índices personalizados** y de texto completo

BD de Documentos : MongoDB

Modelo de Datos



BD de Documentos : Instalación de MongoDB

- Instalación con Docker

```
docker pull mongo
```

- Levantar el contenedor

```
docker run --name mongo -d -p 27017:27017 mongo
```

- Descargar e instalar MongoDB Compass

New Connection

Manage your connection settings

URI ⓘ

Edit Connection String ☒

mongodb://localhost:27017/

Name

Color

No Color

☐ Favorite this connection

Favoriting a connection will pin it to the top of your list of connections

➤ Advanced Connection Options

How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.

[See example](#)

How do I format my connection string?

[See example](#)

BD de Documentos: Usando Atlas

Crear un servicio de MongoDB en la nube con Atlas

1. Después de configurar la Organización y el Proyecto, proceder a crear un Cluster gratuito.
2. En el apartado **SECURITY\Network Access** agregar 0.0.0.0/0 para permitir acceso desde cualquier IP.

Add IP Access List Entry

Atlas only allows client connections to a cluster from entries in the project's IP Access List. Each entry should either be a single IP address or a CIDR-notated range of addresses. [Learn more](#)

ADD CURRENT IP ADDRESS

ALLOW ACCESS FROM ANYWHERE

Access List Entry:

0.0.0.0/0

Comment:

Permitir Cualquier IP



This entry is temporary and will be deleted in

6 hours



Cancel

Confirm

BD de Documentos: Usando Atlas

Crear un servicio de MongoDB en la nube con Atlas

3. Configurar la conexión con cualquier IDE y probar la conexión.

MongoDB Compass

```
mongodb+srv://<user>:<password>@<host>/
```

DBeaver (jdbc)

```
mongodb+srv://<user>:<password>@<host>/admin?authSource=admin&retryWrites=true
```

BD de Documentos: Estructura de MongoDB

Crear una base de datos

```
use miBaseDeDatos
```

Crear una colección

```
db.createCollection("miColeccion")
```

Crear un documento

```
db.miColeccion.insertOne(  
{nombre: "Juan", edad: 30})
```

MongoDB	SQL
Base de Datos	Base de Datos
Colecciones	Tablas
Documentos	Filas (registros)
Campos	Columnas



BD de Documentos: Estructura de MongoDB

Actividad

- Instalar y configurar un servidor MongoDB.
- Crear una base de datos llamada **ecommerce** y tres colecciones: **productos, clientes y pedidos.**

BD de Documentos : CRUD

Insertar documentos:

Se utiliza los comandos **insertOne()** para insertar un solo documento y **insertMany()** para insertar varios documentos. Ejemplo:

```
db.productos.insertOne({ "nombre": "Laptop", "precio": 1500, "categoria":  
"Electrónica" })
```

```
db.productos.insertMany([ { "nombre": "Smartphone", "precio": 900, "categoria":  
"Electrónica" }, { "nombre": "Audifonos", "precio": 100, "categoria": "Accesorios" }  
)
```

BD de Documentos : CRUD

Actualizar documentos:

Se utiliza **updateOne()**, **updateMany()**, y operadores como **\$set** y **\$inc** de la siguiente manera:

```
db.productos.updateOne({ "nombre": "Laptop" }, { $set: { "precio": 1400 } })  
db.productos.updateMany({ "categoria": "Electrónica" }, { $inc: { "precio": 100 } })
```

El operador **\$inc** incrementa o decrementa el valor numérico de todos los elementos que coinciden con el filtro aplicado.

BD de Documentos : CRUD

Actividad

- Insertar algunos productos con diferentes categorías y precios.
- Utilizar como ejemplo el archivo "productos.json" proporcionado por el docente.
- Incrementa en 10% el descuento de los productos de categoría "hogar".
- Agregar un nuevo campo a los televisores: {SistemaOperativo : "Android TV"}

BD de Documentos : CRUD

Eliminar documentos:

Se utiliza `deleteOne()` y `deleteMany()` de la siguiente manera :

```
db.productos.deleteOne({ "nombre": "Audifonos" })  
db.productos.deleteMany({ "categoria": "Accesorios" })
```

BD de Documentos : CRUD

Filtrado de datos:

Se utiliza `find()`, `findOne()` para aplicar filtros.

```
db.productos.find({ "categoria": "Electrónica" })  
db.productos.findOne({ "nombre": "Laptop" })
```

BD de Documentos : CRUD

Proyección de campos:

Si solo desea mostrar algunos campos se le agrega un segundo parámetro al find:

```
db.productos.find({}, {"nombre": 1, "categoria": 1 })
```

```
db.productos.find({ "categoria": "Electrónica" }, { "stock": 0 })
```

- Esto último excluye “stock” pero muestra todos los demás campos.

BD de Documentos : CRUD

Operadores lógicos y de comparación:

Los operadores se utilizan dentro de un objeto de consulta en el método find para especificar criterios de búsqueda. Se puede combinar múltiples operadores para crear consultas más complejas

Ejemplo 1: consulta productos que son electrónicos o cuyo precio es mayor a 500.

```
db.productos.find({
  $or: [
    { categoria: "Electrónica" },
    { precio: { $gt: 500 } }
  ]
})
```

Ejemplo 2: Consulta productos cuya categoría sea "Electrónica" o "Hogar".

```
db.productos.find({
  categoria: {
    $in: ["Electrónica", "Hogar"]
  }
})
```

BD Clave-Valor: Comandos REDIS

Operadores lógicos y de comparación:

Operador	Descripción	Ejemplo de uso
\$eq	Igual a.	{ "edad": { "\$eq": 30 } }
\$ne	No igual a.	{ "edad": { "\$ne": 30 } }
\$gt	Mayor que.	{ "edad": { "\$gt": 30 } }
\$gte	Mayor o igual que.	{ "edad": { "\$gte": 30 } }
\$lt	Menor que.	{ "edad": { "\$lt": 30 } }
\$lte	Menor o igual que.	{ "edad": { "\$lte": 30 } }
\$in	Está en una lista de valores.	{ "edad": { "\$in": [25, 30, 35] } }
\$nin	No está en una lista de valores.	{ "edad": { "\$nin": [25, 30, 35] } }
\$and	Lógica "y" para combinar condiciones.	{ "\$and": [{ "edad": { "\$gt": 20 } }, { "edad": { "\$lt": 40 } }] }
\$or	Lógica "o" para combinar condiciones.	{ "\$or": [{ "edad": { "\$lt": 20 } }, { "edad": { "\$gt": 40 } }] }
\$not	Niega una condición.	{ "edad": { "\$not": { "\$gt": 30 } } }
\$exists	Verifica si un campo existe.	{ "nombre": { "\$exists": true } }

BD de Documentos : CRUD

Actividad

- Mostrar los productos sin stock
- Mostrar los productos con calificación mayor a 4.5
- Buscar productos con precio entre 700 y 900

BD de Documentos : Comandos Avanzados

Ordenar:

//Ordenar de menor a mayor

```
db.productos.find({ categoria: "Electrónica" }).sort({ precio: 1 })
```

//Ordenar de mayor a menor

```
db.productos.find({ categoria: "Electrónica" }).sort({ precio: -1 })
```


BD de Documentos : Comandos Avanzados

Agregaciones:

//Contar todos los documentos de una colección.

```
db.productos.aggregate({$count: "total"})
```

//Contar todos los productos de una categoría específica.

```
db.productos.aggregate([
  { $match: { categoria: "Electrónica" } },
  { $count: "total" } ])
```

//Contar el total productos y sumar los precios agrupados por categoría.

//Mostrar el resultado ordenado por el total.

```
db.productos.aggregate([
  {$group: { _id: "$categoria",
             conteo: { $sum: 1 },
             sumaPrecio: { $sum: "$precio" } } },
  {$sort: {conteo: 1}} ])
```

BD de Documentos : Comandos Avanzados

Consultas en documentos anidados:

Los documentos pueden contener otros documentos o arrays como parte de su estructura. Para consultar información dentro de documentos anidados se usa **dot notation**.

```
db.productos.insertOne({
  "nombre": "Laptop",
  "categoria": "Electrónica",
  "detalles": {
    "fabricante": "Lenovo",
    "modelo": "XYZ-2019",
    "garantia": "2 años"
  })

//Consultar dentro de documentos anidados.
db.productos.find({ "detalles.fabricante": "Lenovo" })
```

BD de Documentos : Comandos Avanzados

Consulta de elementos de un array:

Busca documentos que contengan un valor específico dentro de un array.

```
//Consulta productos que tengan "Pantalla táctil" en su lista de características.
```

```
db.productos.find({  
  características: "Pantalla táctil"  
})
```

```
//Encuentra productos cuyo primer elemento en el array "características"
```

```
//sea "Cámara de 64MP".
```

```
db.productos.find({  
  "características.0": "Cámara de 64MP"  
})
```

BD de Documentos : Comandos Avanzados

Consultas de rango de fechas:

Ejemplo: Consulta productos lanzados entre el 1 de enero de 2023 y el 1 de enero de 2024.

```
db.productos.find({  
  fecha_lanzamiento: {  
    $gte: ISODate("2023-01-01"),  
    $lt:  ISODate("2024-01-01")  
  }  
})
```

BD de Documentos : Búsqueda Textual

MongoDB soporta la búsqueda de texto completo con índices textuales. Para habilitarlo, necesitas crear un índice textual (text)

```
db.productos.createIndex({ descripcion: "text" })
```

Consulta: Busca productos que contengan la palabra “Smart” en la descripción.

```
db.productos.find({ $text: { $search: "Smart" } })
```

```
db.productos.find({ $text: { $search: "táctil led 4k" } })
```

Ordenar por relevancia: encontrar los documentos más relevantes que coincidan con todas las palabras clave proporcionadas.

```
db.productos.find({ $text: { $search: "pulgadas" } })  
    .sort({ score: { $meta: "textScore" } })
```

BD de Documentos : Comandos Avanzados

Actividad

- Consultar el número total de productos agrupados por marca.
- Mostrar los 10 productos más recientes de acuerdo a su fecha de lanzamiento.
- Buscar los productos cuya descripción mencione RAM y CPU.

Conexión a MongoDB usando PyMongo

- Instalar la librería PyMongo con **conda** o **pip**:
`conda install pymongo`
- Crear un cliente que se conecte a MongoDB.
- Crear o seleccionar una base de datos y una colección.

```
from pymongo import MongoClient
```

```
# Conexión a MongoDB en local
```

```
cliente = MongoClient('mongodb://localhost:27017/')
```

```
# Conexión a MongoDB en DeepNote
```

```
# cliente = MongoClient(os.environ["ATLAS_CONNECTION_STRING"])
```

```
# Crear o seleccionar una base de datos existente
```

```
base_datos = cliente['ecommerce'] # Se crea automáticamente si no existe
```

```
# Crear o seleccionar una colección
```

```
coleccion_productos = base_datos['productos']
```

Operaciones CRUD con Python

Insertar documentos

Insertar un documento en la colección

```
nuevo_producto = {"nombre": "Portátil", "precio": 1500, "stock": 10}
coleccion_productos.insert_one(nuevo_producto)
```

Insertar múltiples documentos

```
productos = [
    {"nombre": "Smartphone", "precio": 800, "stock": 50, "categoria":
"Electrónica"},
    {"nombre": "Sillon", "precio": 1500, "stock": 2, "categoria": "Hogar"},
    {"nombre": "Camisa", "precio": 80, "stock": 50, "categoria": "Ropa"}
]
coleccion_productos.insert_many(productos)
```


Operaciones CRUD con Python

Consultar documentos

Leer un documento específico

```
producto = coleccion_productos.find_one({"nombre": "Portátil"})  
print(producto)
```

Leer todos los documentos con precio mayor a 1000

```
productos_caros = coleccion_productos.find({"precio": {"$gt": 1000}})  
for producto in productos_caros:  
    print(producto)
```

Operaciones CRUD con Python

Actualizar documentos

Actualizar el stock de un producto

```
coleccion_productos.update_one({"nombre": "Portátil"},  
                                {"$set": {"stock": 15}})
```

Actualizar múltiples documentos

```
coleccion_productos.update_many({"precio": {"$lte": 100}},  
                                 {"$set": {"descuento": True}})
```

Operaciones CRUD con Python

Eliminar documentos

Eliminar un documento

```
coleccion_productos.delete_one({"nombre": "Portátil"})
```

Eliminar múltiples documentos con stock bajo

```
coleccion_productos.delete_many({"stock": {"$lt": 5}})
```

Filtros Avanzados y Agregaciones

Productos cuyo precio esté entre 500 y 1000

```
productos_rango = coleccion_productos.find({"precio": {"$gte": 500, "$lte": 1000}})
```

```
for producto in productos_rango:  
    print(producto)
```

Productos que tengan 'Phone' en el nombre usando una expresión regular

```
consulta_regex = {"nombre": {"$regex": "/Phone/", "$options": "i"}} #*
```

```
productos_encontrados = coleccion_productos.find(consulta_regex)
```

```
for producto in productos_encontrados:  
    print(producto)
```

Filtros Avanzados y Agregaciones

Ejemplo de un pipeline de agregación para agrupar productos por categoría y contar cuántos hay en cada categoría:

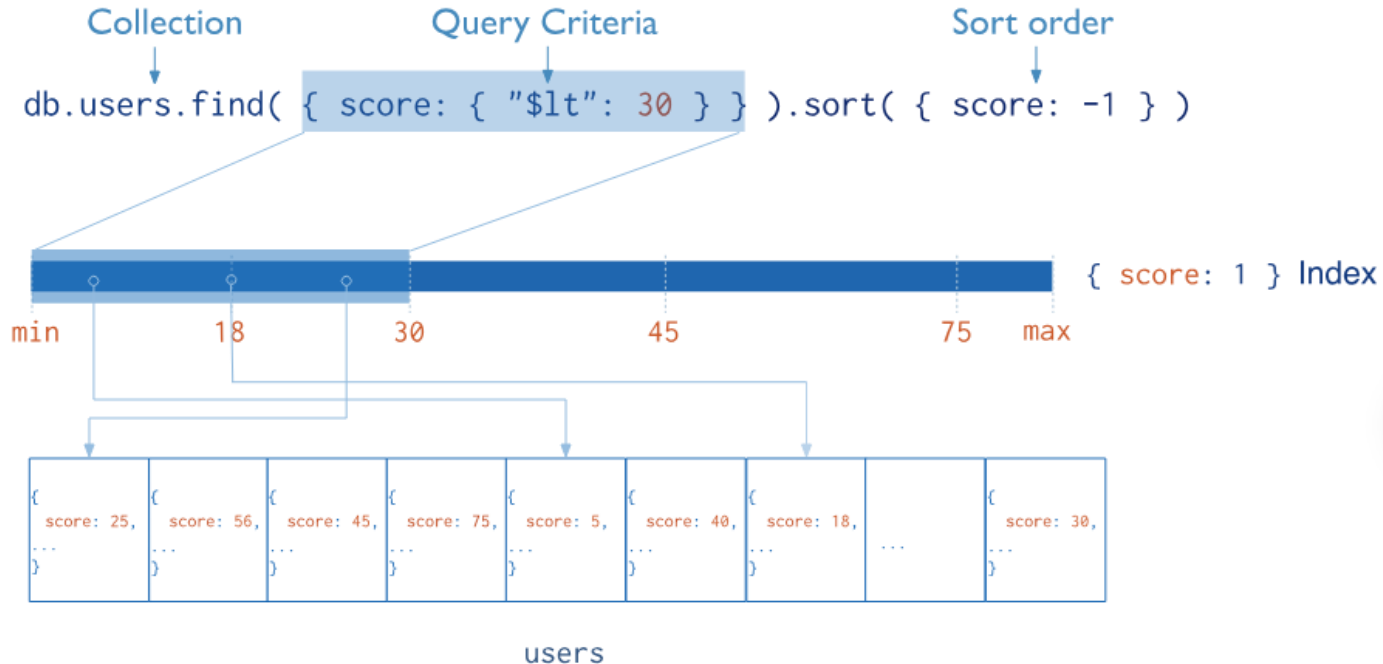
```
# Pipeline de agregación para contar productos por categoría
pipeline = [
    {"$group": {"_id": "$categoria", "total_productos": {"$sum": 1}}},
    {"$sort": {"total_productos": -1}} # Ordenar de mayor a menor
]
resultados = coleccion_productos.aggregate(pipeline)
for resultado in resultados:
    print(resultado)
```

Laboratorio 14



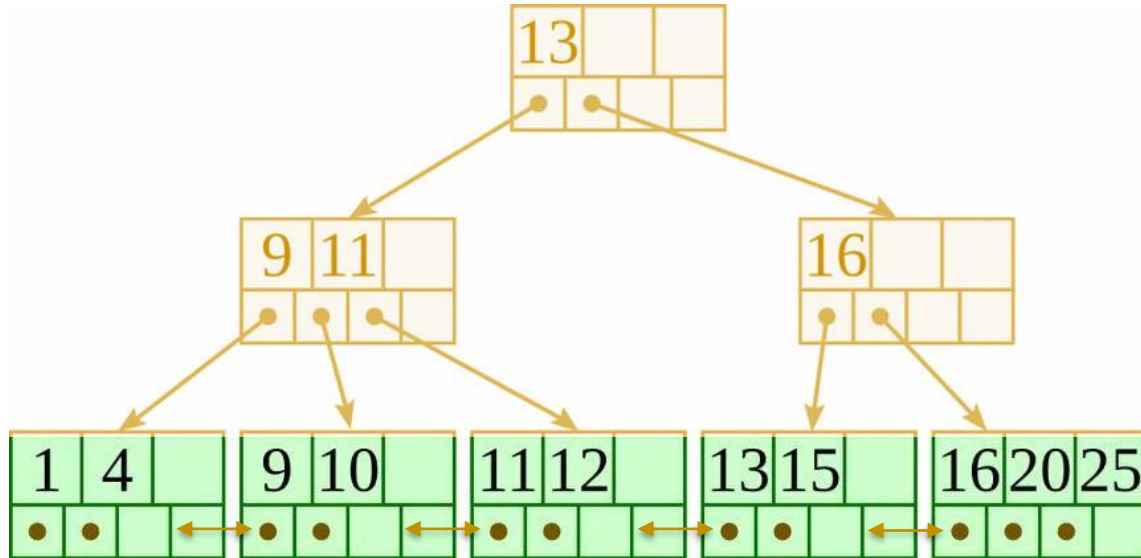
Indexación en MongoDB

La indexación permite a MongoDB organizar los datos para realizar búsquedas más rápidas.



Indexación en MongoDB

MongoDB implementa principalmente **índices B+ Tree** para la mayoría de sus operaciones de indexación. Este tipo de índice permite búsquedas eficientes y ordenadas, ya que almacena los datos en una estructura de árbol balanceado.



Indexación en MongoDB

Tipos de Índices:

- **Llave primaria:**
 - MongoDB crea automáticamente un índice en el campo `_id` como llave primaria.
- **Índice de un solo campo:**
 - Se crea sobre un solo campo de un documento, permitiendo búsquedas rápidas en ese campo.

```
db.user.createIndex({score: 1 })
```



{ score: 1 } Index

Indexación en MongoDB

Tipos de Índices:

- **Índice compuesto:**
 - Se define sobre múltiples campos, mejorando el rendimiento de consultas que filtran por más de un criterio.

```
// Crear índice
```

```
db.user.createIndex({ userid: 1, score: -1 })
```

```
// Aplicar consultas
```

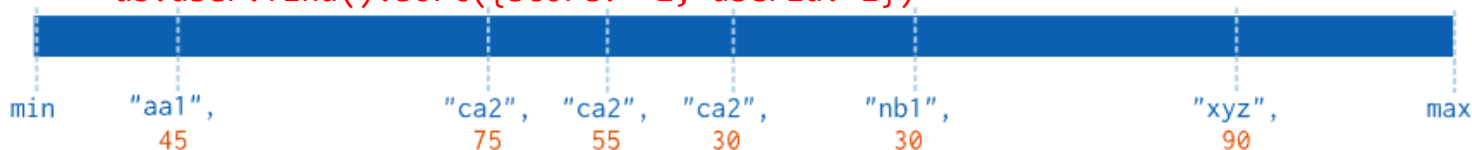
```
db.user.find({ userid: 12345 }).sort({ score: -1 })
```

```
db.user.find({ userid: 12345, score: { $lt: 50 } })
```

```
db.user.find().sort({ userid: 1, score: -1 })
```

```
db.user.find({score: { $lt: 50 }, userid: 12345 })
```

```
db.user.find().sort({score: -1, userid: 1})
```



{ userid: 1, score: -1 } Index

collection



Indexación en MongoDB

Tipos de Índices:

- **Índice Multikey:**
 - MongoDB permite la creación de índices para arrays o campos embebidos.

// Crear índice

```
db.user.createIndex({ "addr.zip": 1 })
```

// Aplicar consultas

```
db.user.find({ "addr.zip": "78610" })
```

```
db.user.find().sort({ "addr.zip": 1 })
```

```
db.user.find({ "addr.zip": { $gte: "10000", $lte: "20000" } })
```

min

"10036"

"78610"

"94301"

max

{ "addr.zip": 1 } Index

collection

```
{
  userid: "xyz",
  addr: [
    { zip: "10036", ... },
    { zip: "94301", ... }
  ],
  ...
}
```

Indexación en MongoDB

Tipos de Índices:

- **Índice de Campo Único (UNIQUE):**
 - Estos índices aseguran que los valores en un campo (o combinación de campos) sean únicos en toda la colección.

```
db.users.createIndex({ dni: 1 }, { unique: true })
```

- Consultar Índices:

```
db.productos.getIndexes()
```

- Eliminar un índice:

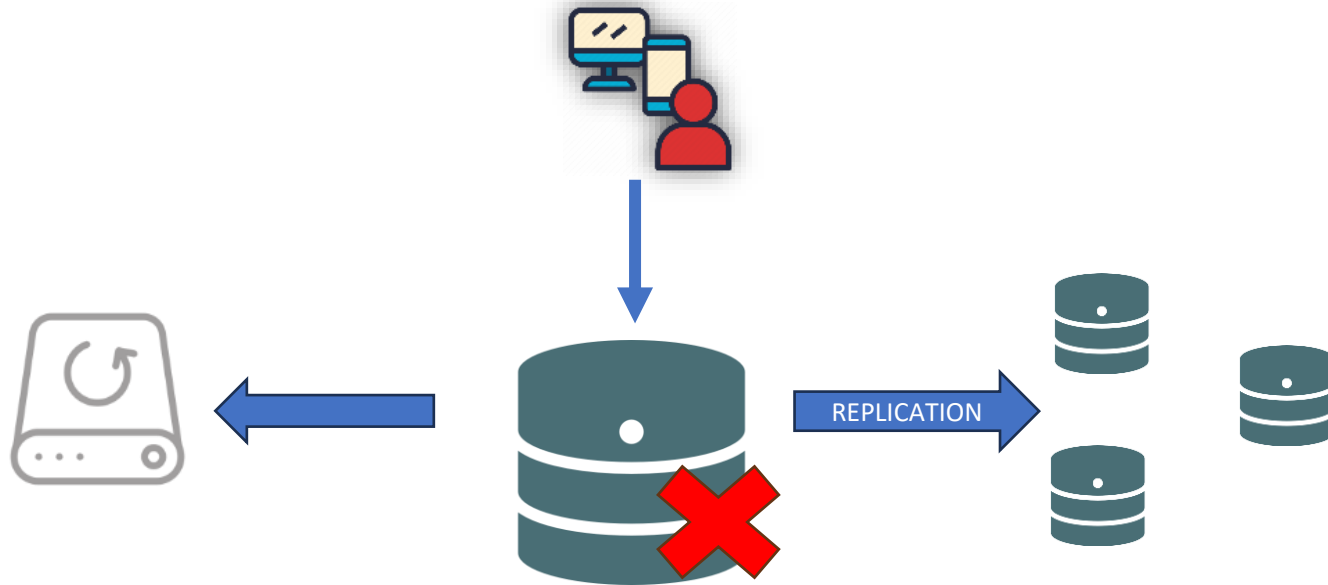
```
db.users.dropIndex("nombre_del_indice")
```

Indexación en MongoDB

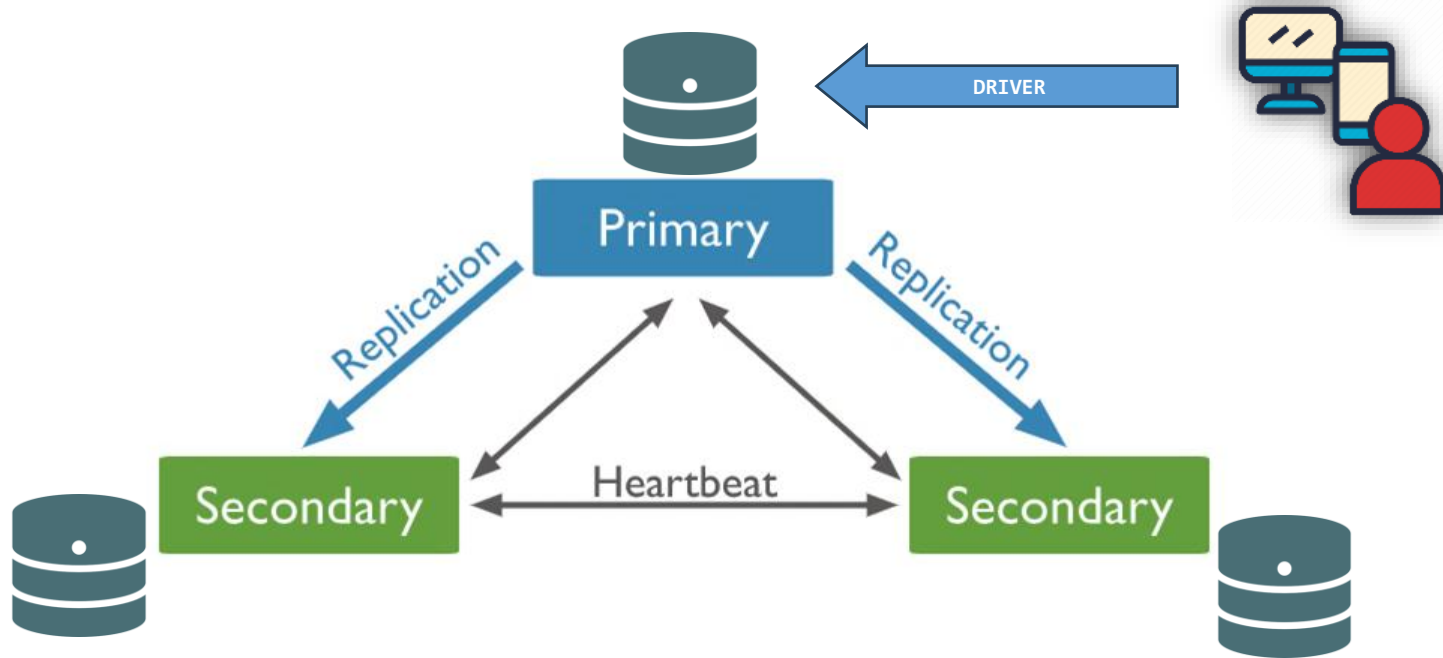
Actividad

- Sobre la tabla de **pedidos y productos** realizar las siguientes consultas frecuentes y proponer una indexación para que una mayor eficiencia:
 - ☐ Buscar pedidos de un cliente en un rango de fechas específico.
 - ☐ Retornar los productos electrónicos lanzados en 2023 con precio entre 500 y 1500.

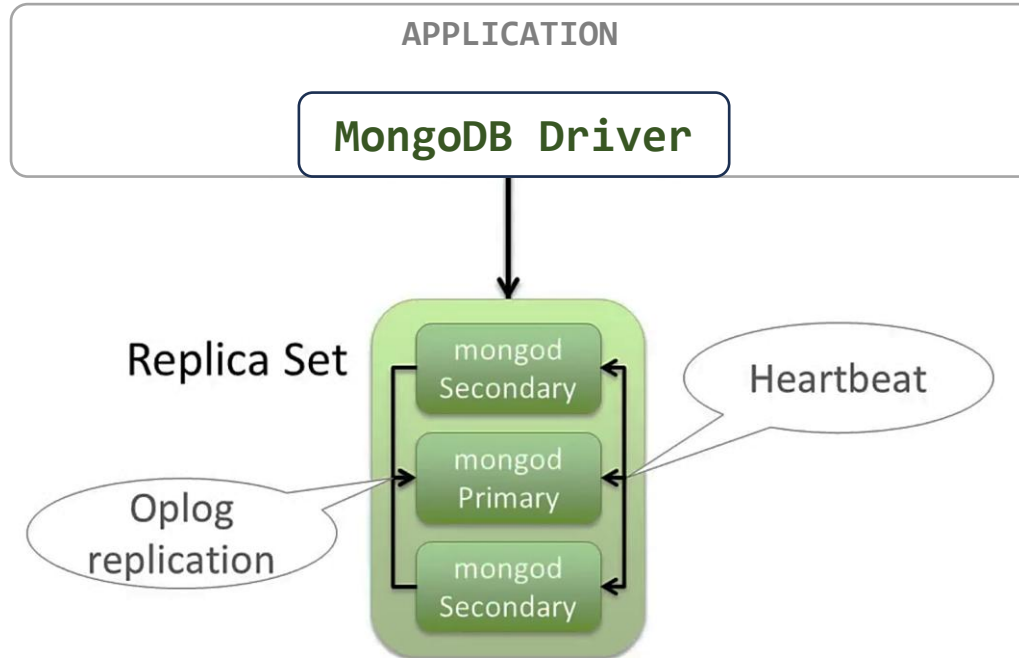
¿Que hacemos ante una falla física del Sistema de BD?



MongoDB: ReplicaSet



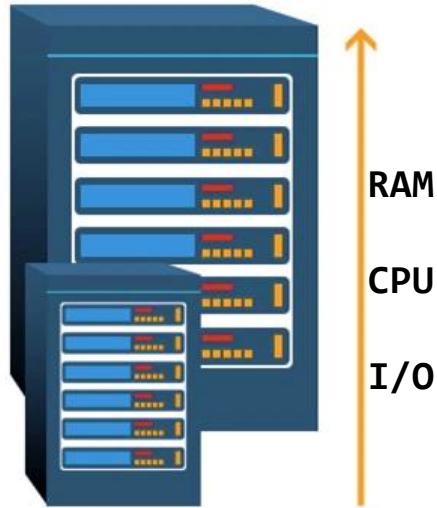
MongoDB: ReplicaSet



MongoDB Distribuido

MongoDB y la
Escalabilidad de Base de Datos

Escalabilidad



Vertical Scaling
(Scaling up)



Horizontal Scaling
(Scaling out)

Escalabilidad: Partición Horizontal



central.server.com

```
{ "name": "Juan Perez", "sede": "Arequipa", "cargo": "Cajero" }  
{ "name": "Ana Vera", "sede": "Trujillo", "cargo": "Secretaria" }  
{ "name": "Felipe Alvarado", "sede": "Lima", "cargo": "Almacenero" }  
{ "name": "Marta Sanchez", "sede": "Trujillo", "cargo": "Cajero" }  
{ "name": "Pedro Torres", "sede": "Lima", "cargo": "Jefe Operaciones" }  
{ "name": "Cecilia Paz", "sede": "Arequipa", "cargo": "Supervisor" }  
{ "name": "Carlos Perez", "sede": "Lima", "cargo": "Supervisor" }  
{ "name": "Maria Briceño", "sede": "Arequipa", "cargo": "Cajero" }
```

```
{ "name": "Juan Perez", "sede": "Arequipa", "cargo": "Cajero" }  
{ "name": "Cecilia Paz", "sede": "Arequipa", "cargo": "Supervisor" }  
{ "name": "Maria Briceño", "sede": "Arequipa", "cargo": "Cajero" }
```



mongo1.server.com

```
{ "name": "Felipe Alvarado", "sede": "Lima", "cargo": "Almacenero" }  
{ "name": "Pedro Torres", "sede": "Lima", "cargo": "Jefe Operaciones" }  
{ "name": "Carlos Perez", "sede": "Lima", "cargo": "Supervisor" }
```



mongo3.server.com

```
{ "name": "Ana Vera", "sede": "Trujillo", "cargo": "Secretaria" }  
{ "name": "Marta Sanchez", "sede": "Trujillo", "cargo": "Cajero" }
```



mongo2.server.com

Escalabilidad: Partición Horizontal



```
{  
  mongo1: {"sede": "Arequipa"},  
  mongo2: {"sede": "Trujillo"},  
  mongo3: {"sede": "Lima"},  
}
```



```
{"name": "Juan Perez", "sede": "Arequipa", "cargo": "Cajero"}  
{"name": "Cecilia Paz", "sede": "Arequipa", "cargo": "Supervisor"}  
{"name": "Maria Briceño", "sede": "Arequipa", "cargo": "Cajero"}
```



mongo1.server.com

```
{"name": "Felipe Alvarado", "sede": "Lima", "cargo": "Almacenero"}  
{"name": "Pedro Torres", "sede": "Lima", "cargo": "Jefe Operaciones"}  
{"name": "Carlos Perez", "sede": "Lima", "cargo": "Supervisor"}
```



mongo3.server.com

```
{"name": "Ana Vera", "sede": "Trujillo", "cargo": "Secretaria"}  
{"name": "Marta Sanchez", "sede": "Trujillo", "cargo": "Cajero"}
```



mongo2.server.com

Escalabilidad: Partición Horizontal



```
{  
  mongo1: {"sede": "Arequipa"},  
  mongo2: {"sede": "Trujillo"},  
  mongo3: {"sede": "Lima"},  
}
```

← {"sede": "Lima"}



```
{ "name": "Juan Perez", "sede": "Arequipa", "cargo": "Cajero" }  
{ "name": "Cecilia Paz", "sede": "Arequipa", "cargo": "Supervisor" }  
{ "name": "Maria Briceño", "sede": "Arequipa", "cargo": "Cajero" }
```



mongo1.server.com

```
{ "name": "Felipe Alvarado", "sede": "Lima", "cargo": "Almacenero" }  
{ "name": "Pedro Torres", "sede": "Lima", "cargo": "Jefe Operaciones" }  
{ "name": "Carlos Perez", "sede": "Lima", "cargo": "Supervisor" }
```



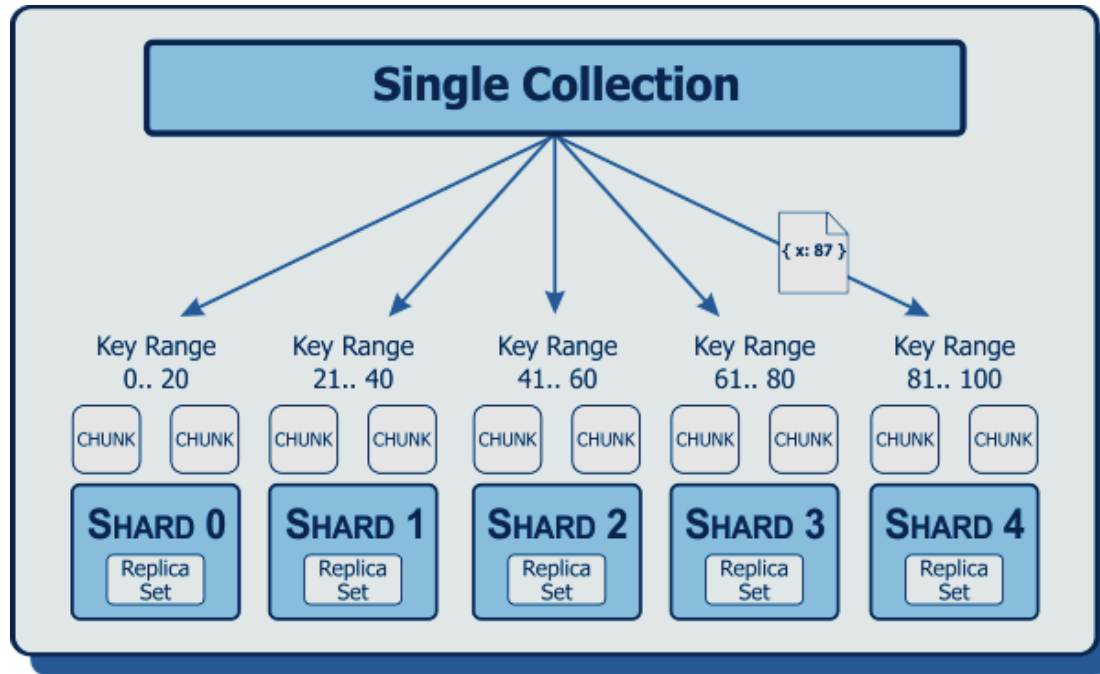
mongo3.server.com

```
{ "name": "Ana Vera", "sede": "Trujillo", "cargo": "Secretaria" }  
{ "name": "Marta Sanchez", "sede": "Trujillo", "cargo": "Cajero" }
```



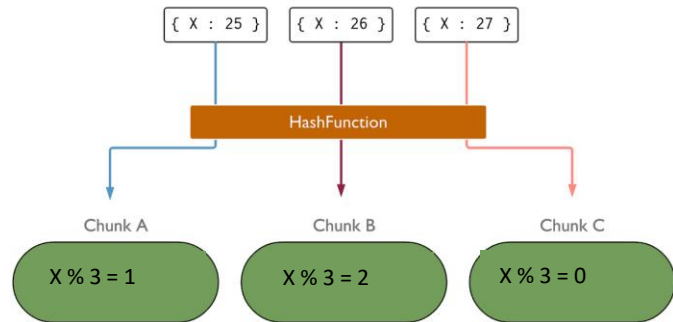
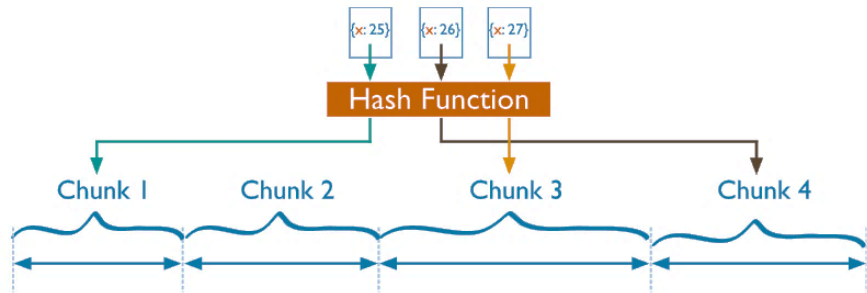
mongo2.server.com

MongoDB: Sharding

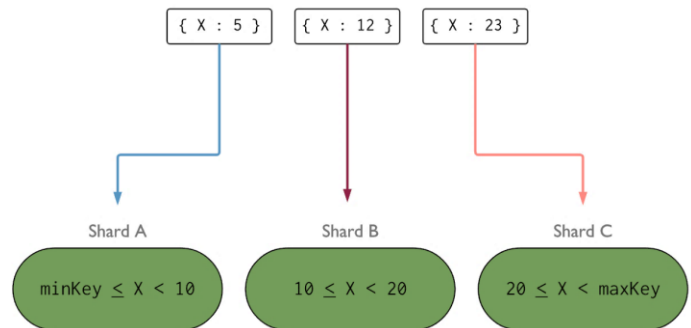
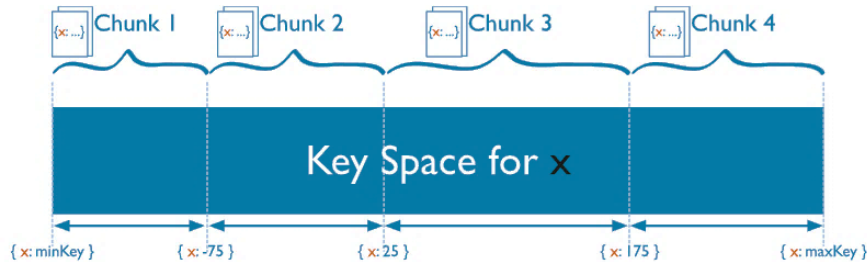


MongoDB: Sharding

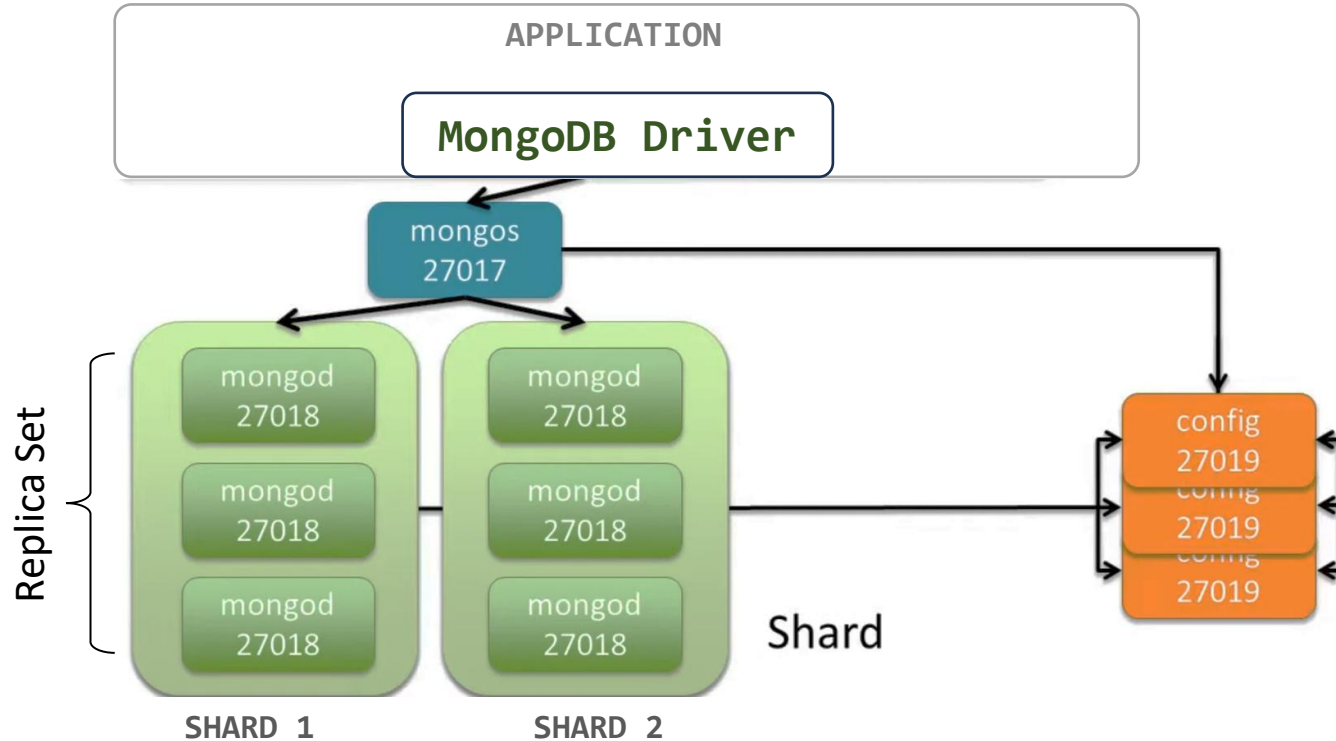
(a) Hashed Sharding



(b) Ranged Sharding



MongoDB: Sharding y ReplicaSet



Laboratorio 14

